Exercise 5

Lazy data, infinite data

Haskell does not evaluate expressions eagerly, but instead does so *lazily*. This opens the door to writing very succint programs that operator on infinite datastructures. We can just write them without needing to have a cut-off and then pick out the pieces we want.

For some more examples, on top of the ones we have here, there's a nice Computerphile video on infinite datastructures, featuring Graham Hutton. There's a nice succint post on some of Haskell's advantages including laziness from serokell.

On with the show.

Simple sieving

Our first go with infinite data is going to be to generate prime numbers. Let's first generate all the integers greater than or equal to 2.

```
candidates :: [Integer]
candidates = [2..]
```

These are our candidate primes. We can now generate prime numbers using the sieve of Eratosthenes. The first element of our candidates is a prime number, we then need to cross off all multiples, and repeat.

Exercise 5.1

Write a function

```
sieve :: [Integer] -> [Integer]
```

that generates all prime numbers.

Hint: Think about using a list comprehension to remove values from your candidate list that are multiples of the current prime.

You'll probably not want to print this list, since Euclid showed that it is infinite in length a while ago.

Instead, to look at (say) the first five elements use

```
take 5 (sieve candidates)
```

Question 5.2

Did you come up with a one-liner using `mod` in your answer? If so, you probably ended up implementing finding primes by trial division.

The prime sieve shouldn't need to do any arithmetic, doing this properly is actually slightly fiddly, if you're interested, this nice paper explains the problem with the one-liner approach.

Don't worry, you're not alone in doing this, if you watched the video with Graham Hutton above, you might have spotted that he does the same thing.

Pythagorean triples again

In Exercise 3, we implemented code to generate all Pythagorean triples with entries less than some integer n.

This time round, we're going to do something a bit smarter and generate an *infinite* list of all *primitive* Pythagorean triples. Recall that a Pythagorean triple is a tuple of three positive integers (a,b,c) such that $a^2+b^2=c^2$, it is called *primitive* if a, b, and c are all coprime.

All such primitive triples can be generated from the root triple $(3,4,5)$ by using matrix generators.

In particular, consider $m>n>0$ with m and n coprime (and one even and the other odd). Then

$$a = m^2-n^2$$
$$b = 2mn$$
$$c = m^2+n^2$$

is a new primitive Pythagorean triple. Writing $\vec{M}=[m,n]$ as a column vector, then the required properties are preserved by premultiplying $\vec{M}$ by any of the three matrices

$$\begin{bmatrix}2 & -1\\ 1 & 0\end{bmatrix}, \begin{bmatrix}2 & 1\\ 1 & 0\end{bmatrix}, \text{ or } \begin{bmatrix}1 & 2\\ 0 & 1\end{bmatrix}.$$

We will use these to generate a tree of all Pythagorean triples. We'll build this up in stages by implementing some helper functions.

Exercise 5.2

First write a function

```
mult :: [[Int]] -> [Int] -> [Int]
```

which computes the result of multiplying a matrix (represented as a list of lists) onto a vector (represented as a list). You may find the functions `zipWith` and `repeat` helpful.

Next up, we need to convert an (m,n) pair into a triple

Exercise 5.3

Write

```
pythTriple :: [Int] -> (Int, Int, Int)
```

which uses the relationship above to generate a triple from a 2-element list. For example

```
Prelude> pythTriple [2, 1]
(3, 4, 5)
Prelude> pythTriple [3, 2]
(5, 12, 13)
```

Finally, this slightly trickier part, generating the tree of all these (m,n) pairs.

Exercise 5.4

Write a function

```
treeGen :: [[Int]] -> [[Int]]
```

This can be written by using the three generator matrices to create a list of the next tree pairs, and then appending the result of a recursive call to `treeGen` to the list.

You may find one of `concatMap` or `concat` useful

```
concatMap :: (a -> [b]) -> [a] -> [b]
concat :: [[a]] -> [a]
```

Hint: be careful that you don’t recurse depth-first down one of the leaves of the tree.

For example, you should see something like

```
Prelude> take 10 (treeGen [[2, 1]])
[[3,2],[5,2],[4,1],[4,3],[8,3],[7,2],[8,5],[12,5],[9,2],[7,4]]
```

Although the order may be different.

Finally, we can create the pythagorean truples by mapping `pythTriple` over the list of pairs obtained from `treeGen`, starting with `[2, 1]`.